

Precisely controlling accessible tools with glob patterns

Two pattern formats

Simple format: matches a tool name or a Bash command.

```
"permissions": {
  "allow": ["Bash(git *)", "Bash(pnpm *)",
    "Read", "Edit"],
  "deny": ["Bash(rm -rf *)", "Bash(sudo *)",
    "Bash(npm publish)"]
}
```

Path patterns: Read, Edit and Write accept the path pattern directly. Do not add `file_path:`; that qualifier is invalid for these tools.

```
"permissions": {
  "deny": [
    "Read(*.env*)",
    "Edit(*.pem)",
    "Write(*.key)",
    "Read(*credentials*)"
  ]
}
```

Where to define permissions

~/.claude/settings.json	(global, all projects)
.claude/settings.json	(project, shared with team)
.claude/settings.local.json	(local machine, gitignore)

Rule evaluation: deny rules from any scope take precedence over ask and allow rules. A local allow cannot override a managed or user-level deny. Use local settings for project-specific preferences within those limits.

Glob syntax

Pattern	What matches
Bash(git *)	Any git command
Bash(npm test*)	npm test , npm test:watch
Read(*.env*)	.env , .env.local , .env.prod
Edit(./src/**)	All files under src/
Write(*.key)	Any .key file

Progressive levels

```
// Level 1 - Auto-approve read tools
{ "permissions": { "allow": ["Read", "Grep", "Glob"] } }

// Level 2 - Git + package manager
{ "permissions": { "allow": ["Read", "Grep", "Glob",
  "Bash(git *)", "Bash(pnpm *)"] } }

// Level 3 - Full development
{ "permissions": { "allow": ["Read", "Grep", "Glob",
  "Edit", "Write", "Bash(git *)", "Bash(pnpm *)"] } }
```

Behavior of the three categories

Category	Behavior
allow	Auto-approved, no prompt
deny	Completely blocked
ask	Confirmation requested
(default)	Default permission mode

Known limitation

A path rule restricts matching tool access; it cannot remove a secret already pasted into a prompt or returned by another tool. Keep sensitive data outside the process access boundary and review shell, MCP and sandbox controls together.